TEXT ANALYTICS

Unsupervised Learning for Text Analytics

Francesco Spinnato

* francesco.spinnato@unipi.it
University of Pisa



Unsupervised Learning for Text Analytics

Learning and Inference (Unsupervised)

Learning (training)

Given training data $\mathbf{X}$, the learning algorithm produces a trained model, $f_{\hat{\boldsymbol{\theta}}}$:

$$\hat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \ \mathcal{L}(\boldsymbol{\theta}; \mathbf{X})$$

where $\mathcal{L}$ is a loss function measuring how well the model is doing.

Inference (application)

The trained model is applied to a (possibly new) input:

$$\hat{\mathbf{z}} = f_{\hat{\boldsymbol{\theta}}}(\mathbf{X})$$

Unsupervised Learning for Text Analytics

Clustering

Clustering

Clustering is an unsupervised learning task whose goal is to assign each document to a cluster.

Given a dataset $\mathbf{X}$ with one row $\mathbf{x}_i$ per document, the model produces:

$$f(\mathbf{X}) = \hat{\mathbf{z}}, \quad \hat{\mathbf{z}} \in \{1, \ldots, K\}^N$$

where K is the number of clusters and $\hat{z}_i$ is the cluster assignment of $\mathbf{x}_i$.

Document Clustering

Clustering partitions a collection into groups, the **clusters**, so that documents in the same cluster are similar to each other and dissimilar to documents in other clusters.

- Nobody provides the groups: they are **discovered**, and it is up to us to interpret them.
- The ingredients come from the previous lecture:
  - documents as TF-IDF vectors, similarity as the **cosine**.

What clustering is used for:

- **exploring** a corpus nobody has read yet
- **grouping search results**: a query for *jaguar* returns pages about the car and pages about the animal
- **finding near-duplicates**: reposted news, copied reviews
- drafting **categories before annotation**

Kinds of Clustering

Two independent choices:

- **Flat** or **hierarchical**: K separate groups, or a tree of nested groups.
- **Hard** or **soft**: every document in exactly one cluster, or every document in several clusters to different degrees.

Method	Flat / hierarchical	Hard / soft
k-means	flat	hard
Agglomerative clustering	hierarchical	hard
Topic models	flat	soft

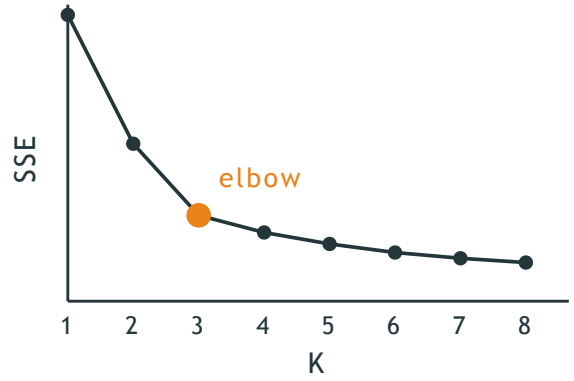
K-Means

Fix the number of clusters K. Each cluster S_j is represented by its **centroid** $\boldsymbol{\mu}_j$, the mean of its documents.

1. **Initialize**: pick K starting centroids, e.g. K random documents.
2. **Assign**: put every document in the cluster of its nearest centroid.
3. **Update**: move every centroid to the mean of its documents:
   $\boldsymbol{\mu}_j = \frac{1}{|S_j|} \sum_{\mathbf{x} \in S_j} \mathbf{x}$
4. **Repeat** 2 and 3 until no document changes cluster.

K-means minimizes the within-cluster **sum of squares**:

$$\sum_{j=1}^{K} \sum_{\mathbf{x} \in S_j} \|\mathbf{x} - \boldsymbol{\mu}_j\|^2$$



K-Means on Text

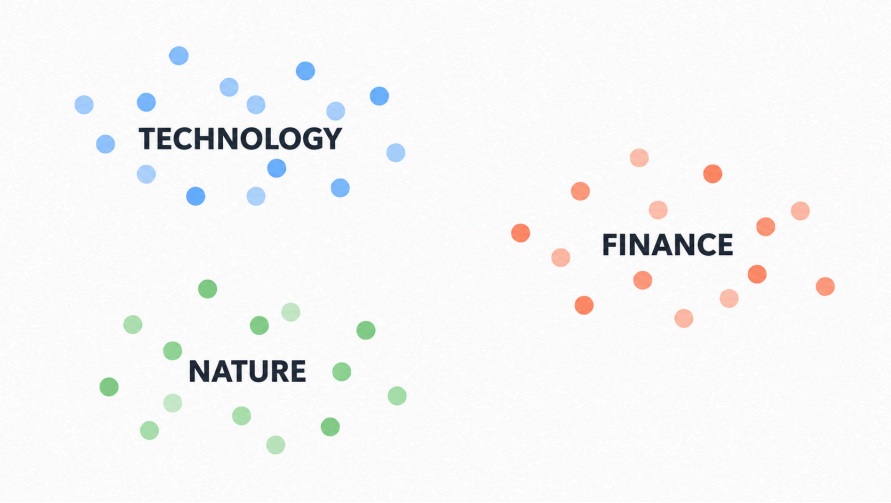
Euclidean distance or cosine? K-means is defined with Euclidean distance, text similarity with the cosine. Normalize every TF-IDF vector to length 1 first, and the two agree:

$$\|\mathbf{a} - \mathbf{b}\|^2 = 2 - 2\cos(\mathbf{a}, \mathbf{b}) \qquad \text{when } \|\mathbf{a}\| = \|\mathbf{b}\| = 1$$

Centroids describe their clusters.

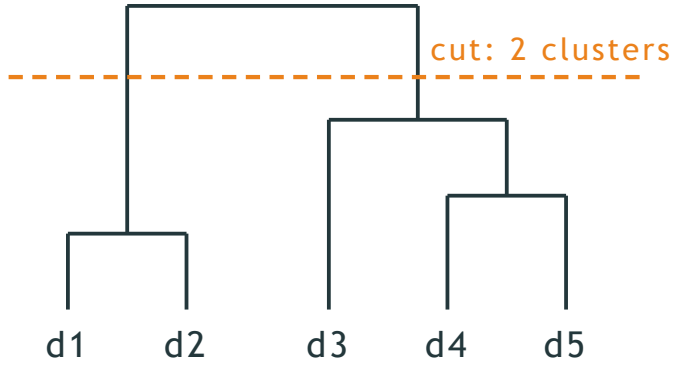
- A centroid is a vector over the vocabulary: it is a readable description for the cluster.

Choosing K. Look for the point where adding clusters stops helping much (the **elbow**), and in the end read the clusters.



Hierarchical (Agglomerative) Clustering

1. Start with every document in a cluster of its own.
2. **Merge** the two closest clusters.
3. Repeat until a single cluster is left.



"Closest" between two clusters is the **linkage**:

- **single**: the closest pair of documents
- **complete**: the farthest pair
- **average**: the mean over all pairs

The result is a **dendrogram**.

- Cutting it at some height gives a flat clustering, without fixing K in advance.
- Needs N^2 pairwise similarities: fine for thousands of documents, not millions.

Other Clustering Methods

Once documents are **vectors**, a corpus is a table of numbers, and **any clustering method for tabular data applies**:

- **k-medoids**: like k-means, but each cluster is represented by an actual document (the medoid) instead of a mean; works with any distance, e.g. cosine.
- **DBSCAN**: grows clusters from dense regions; finds K by itself and leaves isolated documents out as **noise**.

The **shape of the table** matters: in a very high-dimensional, sparse space, distances become less informative and "density" is hard to define.

- Cosine on normalized TF-IDF usually works better than raw Euclidean distance.

Evaluating Clustering

There are no labels, so there is no single right answer to compare against. Three complementary checks:

- **Internal**: look only at the clusters themselves. The **silhouette score** asks, for each document, whether it is closer to its own cluster than to the nearest other one. It ranges from -1 (probably in the wrong cluster) to 1 (well placed), and averaging it over the corpus gives another way to **choose** K.
- **External**: if some labels exist (e.g. news sections), check how well the clusters match them. The labels were not used to build the clusters.
- **Human**: read the top terms of each centroid and a sample of documents. Are the clusters meaningful **for our purpose**?

From Clusters to Topics

Hard clustering puts every document in **exactly one** group. But text can be complex and rarely cooperates:

- A phone review discusses the **battery**, the **camera** and the **price**. Which single cluster does it belong to?
- Words are shared across themes: *star* belongs to astronomy and to cinema, *bank* to finance and to rivers.

We want two things clustering methods cannot give:

- a document described as a **mixture** of themes, in proportions
- each theme described by **many words with weights**, not by a single centroid

That is what **topic models** do.

Unsupervised Learning for Text Analytics

Topic Modeling

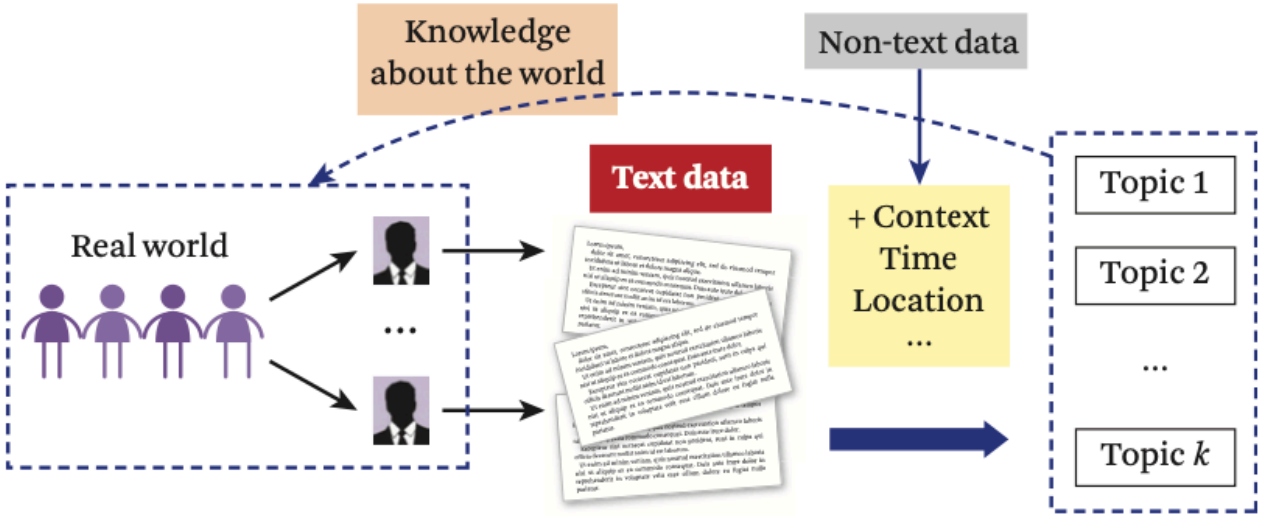
What Is a Topic?

There is no single formal definition, but intuitively a topic is the main idea discussed in text data: the **theme or subject** of a conversation or of a text.

- e.g. genetics, biology, computer science, politics, … in news articles
- but also music and movie genres, e.g. established by song lyrics, etc.

Topics describe some knowledge about the world and can have different **granularities**, e.g.,

- a book
- a chapter
- an article
- a sentence, …



What Can Thematic Knowledge Be Used For?

To organize and browse documents

- the Dewey Decimal Classification (used for libraries since 1876): new books are added to a library in their appropriate location based on subject
- for information retrieval, e.g. to find all the journal articles in a collection covering a topic we are interested in

For text mining

- What are the main trending topics on Reddit today? How do they change over time and location?
- What are the three main research topics today?
- For a given product, what are the most recurrent topics in the negative and in the positive reviews?

Topic Modeling

Topic models provide an automated procedure to organize, understand and summarize large collections of unstructured textual information.

- They require **no labels**: only the documents and the number of topics k.

Topic modeling helps in:

- **discovering** hidden topical patterns in the collection, through statistical regularities
- **annotating** documents according to these topics
- **using** these annotations to organize, search and summarize documents

Topic models are an example of **mixture models**:

- documents can contain multiple topics
- words can belong to multiple topics

Topic Models as Language Models

A **unigram language model**, the simplest language model, is a probability distribution over the words of a vocabulary.

Topic models can be seen as language models where:

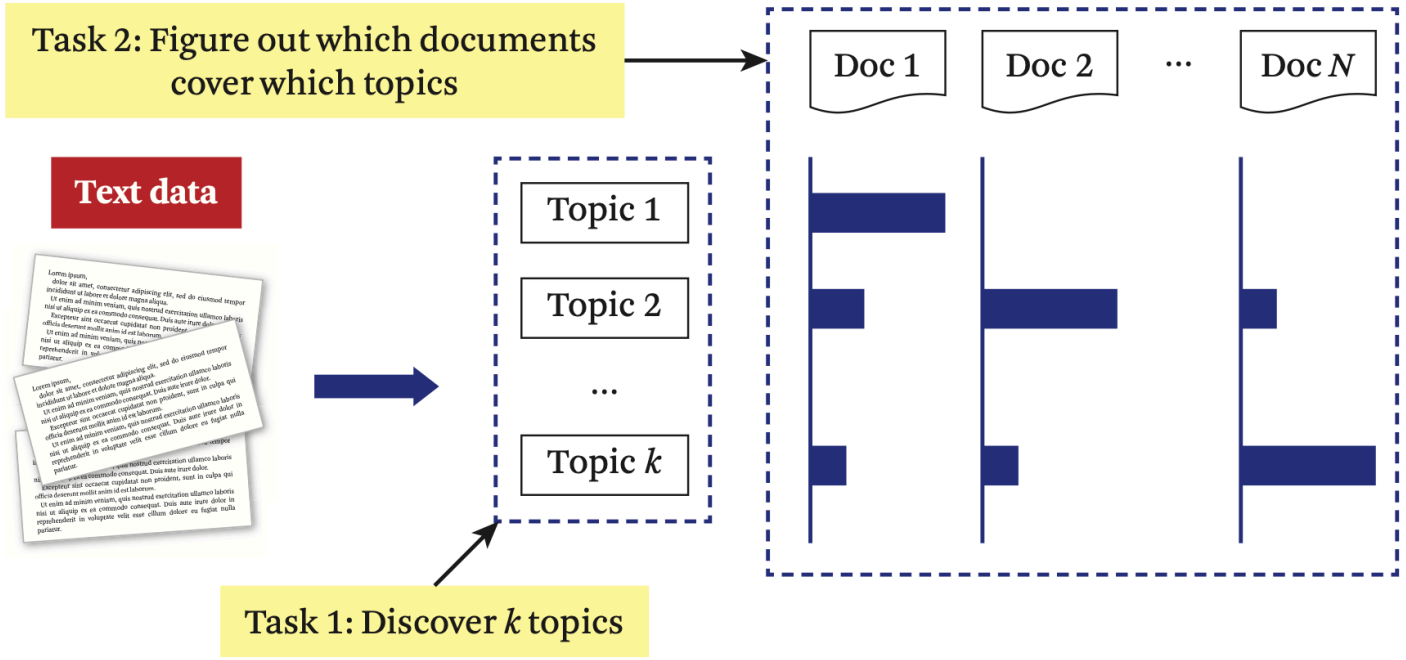
- a separate unigram distribution is estimated for each topic
- each document is represented as a **mixture** of multiple topics

A topic is a probability distribution over a fixed vocabulary of words.

The Topic Mining Task

A topic model **simultaneously estimates two sets of probabilities:**

1. the probability of observing each **word** for each **topic**
2. the probability of observing each **topic** in each **document**



The Topic Mining Task

A topic model outputs, for each document d, represented as a vector $\mathbf{x}_d$, a vector of **topic proportions** over k topics:

$$f(\mathbf{x}_d) = \boldsymbol{\pi}_d = [\pi_{d1}, \ldots, \pi_{dk}], \quad \pi_{dj} \geq 0, \quad \sum_{j=1}^{k} \pi_{dj} = 1$$

Example: a phone review: $[\overbrace{0.5}^{\text{battery}}, \overbrace{0.3}^{\text{camera}}, \overbrace{0.2}^{\text{price}}]$. If we need a hard assignment (more similar to clustering), we take the dominant topic:

$$\hat{z}_d = \underset{j \in \{1,\ldots,k\}}{\text{argmax}} \ \pi_{dj}$$

The topics are **not given**: the model also has to discover what each topic j is.

The Topic Mining Task

Two different tasks (or subtasks):

- **Task 1**: discover the k topics from a collection of text.
- **Task 2**: figure out which documents cover which topics, and to what extent.

Input

- a collection of N text documents d_i: $C = \{d_1, \ldots, d_N\}$
- number of topics: k

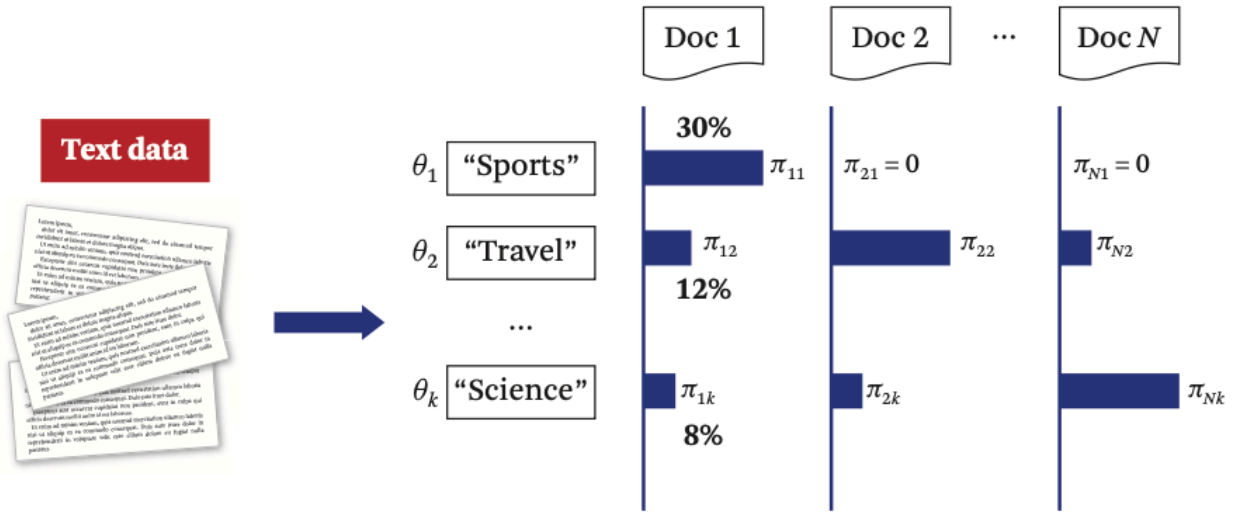
Output

- k topics: $\{\theta_1, \ldots, \theta_k\}$
- coverage of topics in each document d: $\{\pi_{d1}, \ldots, \pi_{dk}\}$, with $\sum_{j=1}^{k} \pi_{dj} = 1$
- π_{dj} = probability of d covering topic θ_j

The coverages sum to one, so a document cannot cover topics outside the ones we discovered. **But how do we define a topic θ_j?**

A Term as a Topic

- A topic can be viewed as a **term** (word or phrase), such as *"sports"* or *"travel"*, representing a recurring theme in text data.
- The task involves identifying k representative terms from the corpus,
  - to find words that are **interesting**,
  - but also **distinctive**, non-overlapping, e.g. not both *sport* and *football*.
  - avoiding overly common terms (e.g. stopwords)



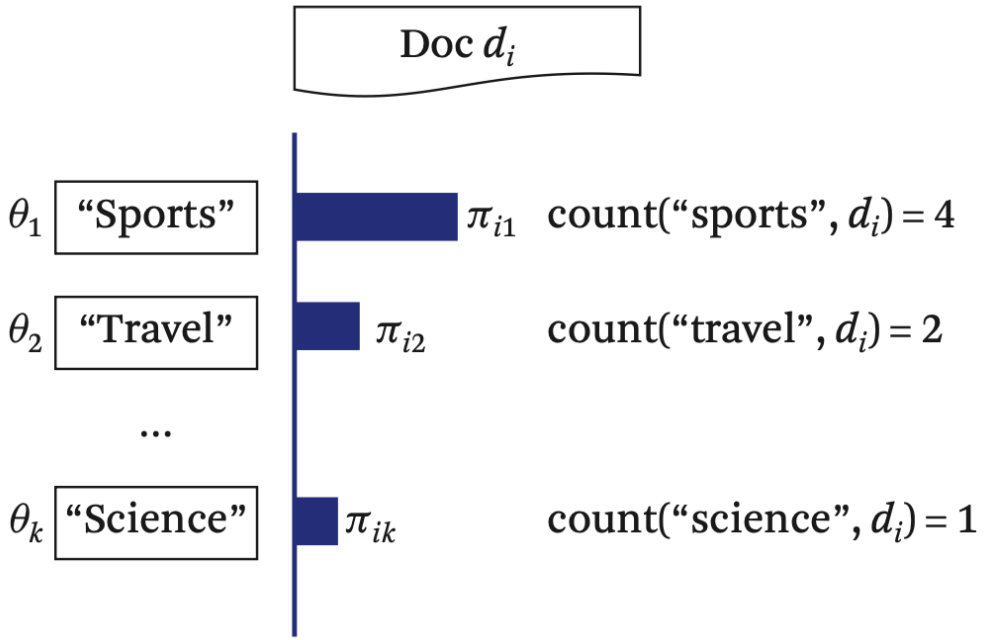
Step 1: Topic Discovery

1. Identify **candidate topical terms**, e.g. all the terms in the corpus (simplest case), or a set of words/phrases.
2. Design a **scoring function** to favor terms that are frequent *and* representative, i.e. terms that can represent a lot of content in the collection.
   - raw frequency is not a good idea (functional words, determiners, stopwords)
   - better to use TF-IDF scoring, or domain-specific heuristics like number of hashtags, news titles, …
3. **Discover the k topical terms.**
   - pick the k terms with the highest scores (simple, but words may be similar)
   - better to use **Maximal Marginal Relevance (MMR)** re-ranking: choose terms according to their rank *and* their dissimilarity with higher-ranking terms

Step 2: Analyze Topic Coverage

1. Count the occurrences of each topical term in each document.
2. Normalize the counts so that they sum to 1 in each document.

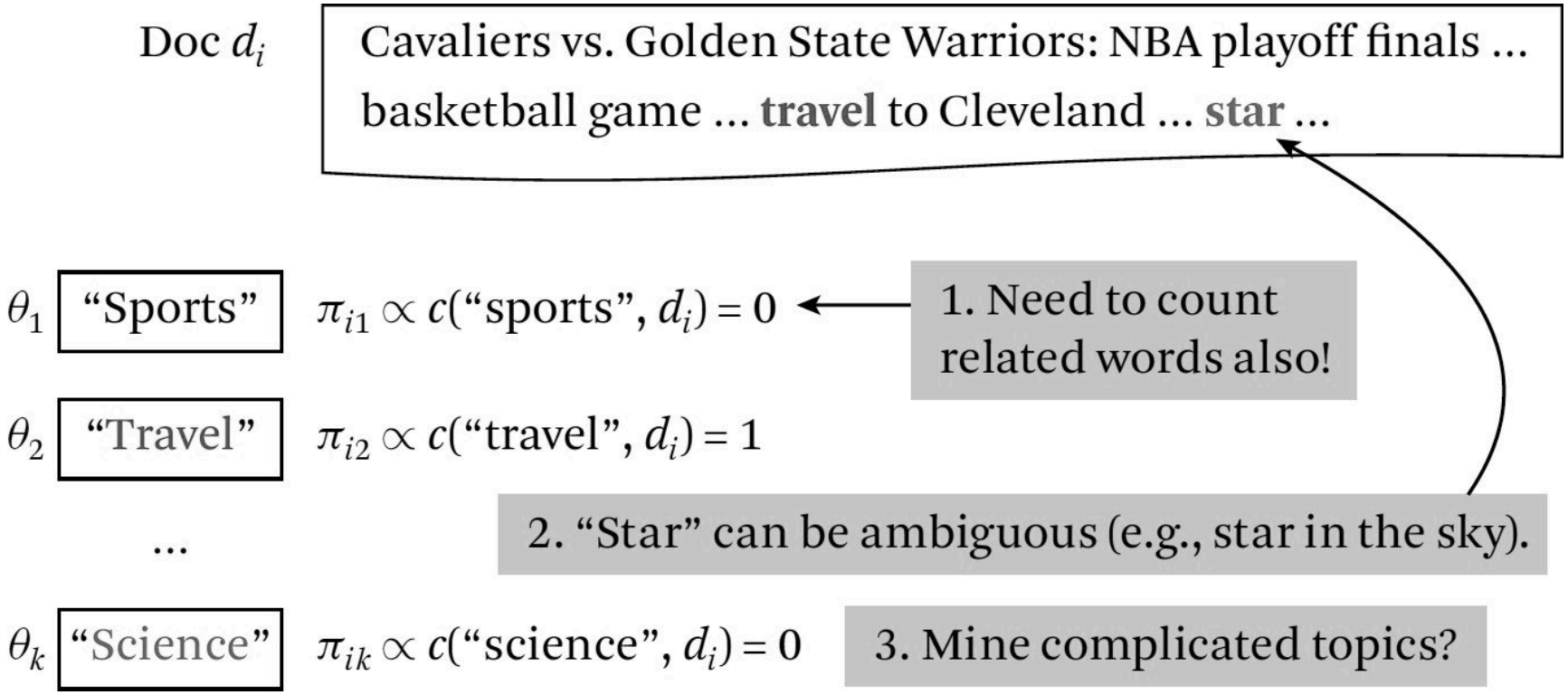
π_{dj} is the proportion of topic θ_j within document d:



Computing topic coverage when a topic is a term.

$$\pi_{ij} = \frac{\text{count}(\theta_j, d_i)}{\sum_{L=1}^{k} \text{count}(\theta_L, d_i)}$$

A Term as a Topic: Problems



A Term as a Topic: Take-away Problems

Vocabulary coverage

- The relation between different terms and a topic is not represented.
- The topical term (e.g. *science*) may never occur, while related words may be very frequent (e.g. *microscope*, *researcher*).

Polysemy

- A term can be ambiguous (e.g. *star*).

Expressive power

- Can only represent simple, general topics.
- Specialized or complex topics (e.g. *mechanical doping in professional cycling*) are difficult to represent with single simple terms.

Topic as a probability distribution

"Topics as terms" describes a topic with a single term, or a set of terms. That struggles with nuanced topics and with ambiguity.

"Topics as word distributions" removes both limits: a topic is a **distribution over words**.

- This allows the model to capture complex and nuanced themes, using groups of related words and their relative weights.
- **Ambiguous words** can belong to multiple topics with different probabilities.
- This representation is the basis of topic models such as **PLSA** and **LDA**.

Topic as a probability distribution

Each document is a mixture of topics, as before. What changes is the topic: no longer a single word, but a distribution over words.

We imagine each word of the document is produced in two steps: pick a topic according to the document's coverage, then pick a word from that topic.

The probability of a word in document d is a **weighted sum** over the topics:

$$P(w \mid d) = \sum_{j=1}^{k} \pi_{dj} \, P(w \mid \theta_j)$$

where the weights π_{dj} are the coverages of document d: how much of d is about topic j. A model that describes how the data could have been produced like this is a **generative model**.

Topic as a probability distribution

$$P(w \mid d) = \sum_{j=1}^{k} \pi_{dj} \, P(w \mid \theta_j)$$

What changes with respect to topics as terms? The formula is the same; what changes is $P(w \mid \theta_j)$.

- **Topic as a term:** $P(w \mid \theta_j)$ is 1 for the topical term and 0 for every other word. So $P(w \mid d) = 0$ for any word that is not one of the k topical terms: *microscope* tells us nothing about *science*.
- **Topic as a distribution:** $P(w \mid \theta_j)$ spreads over the whole vocabulary, so *microscope* and *researcher* get some probability under the science topic, and *star* can have weight in both astronomy and cinema.

Topic as a probability distribution: Example

Suppose the model has learned (we don't know how yet) that d is 70% *science* and 30% *sports*. We see *microscope* in d but do not know which topic produced it, so we sum over both:

$$P(microscope \mid d) = 0.7 \cdot \underbrace{0.03}_{P(microscope|\text{science})} + 0.3 \cdot \underbrace{0.001}_{P(microscope|\text{sports})} = 0.0213$$

With **topics as terms**, each topic produces only its own word, and *microscope* is neither the word *science* nor the word *sports*, so it is ignored, even though it is clearly about science:

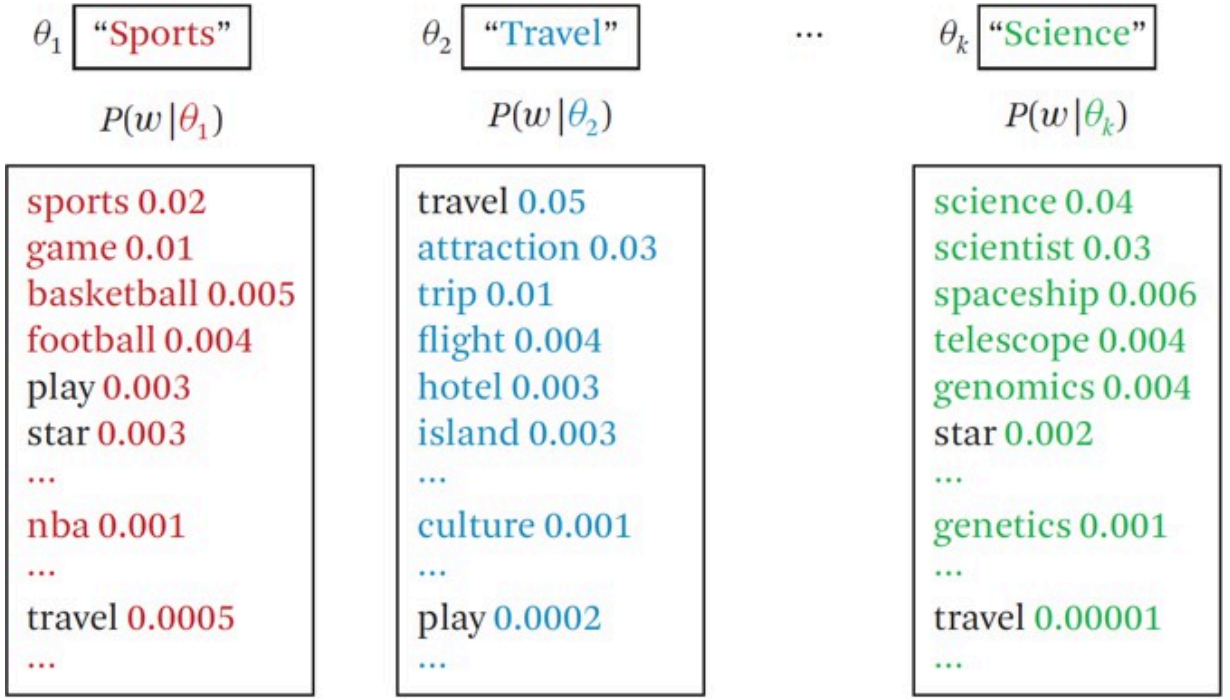
$$P(microscope \mid d) = 0.7 \cdot 0 + 0.3 \cdot 0 = 0$$

In words: given that d is 70% about science and 30% about sports, and given how likely *microscope* is in each topic, $P(microscope \mid d)$ is the probability that the next word d produces is *microscope*.

Topics as Word Distributions (i.e. as Unigram Models)

For every topic, we have a **word distribution over all the words in the vocabulary.**

- If we sample words from this distribution, we tend to see words related to this topic.
- Words unrelated to the topic still get a small, **non-zero** probability.
- Some words are **shared** by more than one topic, with a reasonably high probability, e.g. *travel*, *star*, *play*.



$\sum_{w \in V} p(w|\theta_i) = 1$ Vocabulary set: $V = \{w_1, w_2, \ldots\}$

The Topic Mining Task, with Word Distributions

Input

- a collection of N text documents d_i: $C = \{d_1, \ldots, d_N\}$
- number of topics: k
- **vocabulary**: $V = \{w_1, w_2, \ldots\}$

Output

- k topics $\{\theta_1, \ldots, \theta_k\}$, **each of which is now a word distribution**: $\sum_{w \in V} P(w \mid \theta_j) = 1$
- coverage of topics in each document d: $\{\pi_{d1}, \ldots, \pi_{dk}\}$, the topic coverage distribution for each document
- π_{dj} = prob. of d covering topic θ_j

So the output is now **two families of probability distributions**.

Topics $\theta_1, \ldots, \theta_k$ and the coverages π_{dj} can be estimated in several ways: the Expectation-Maximization algorithm for PLSA, Gibbs sampling for LDA. We only see the intuition of the latter.

Probabilistic Latent Semantic Analysis (PLSA)

Probabilistic Latent Semantic Analysis (**PLSA**) models every document as a mixture of $k+1$ **unigram language models**:

- k **topic** models $\theta_1, \ldots, \theta_k$, the ones we want to discover
- one **background** model θ_B, shared by all documents, which absorbs the common words (*the*, *of*, *and*) so they do not end up in every topic

Two key assumptions:

- a topic can be represented as a **word distribution**
- a text document is a **sample of words** drawn from a probabilistic model

PLSA: Generating Text with Multiple Topics

Generating a word takes **two steps**.

1. **Choose a component model**, controlled by:
   - λ_B: the probability of choosing the background model θ_B
   - π_{dj}: the probability of choosing topic θ_j, if we did *not* choose the background, with $\sum_{j=1}^{k} \pi_{dj} = 1$
2. **Draw a word** from the selected distribution.

$$P(w \mid d) = \underbrace{\lambda_B \, P(w \mid \theta_B)}_{\text{background model}} + \underbrace{(1 - \lambda_B) \sum_{j=1}^{k} \pi_{dj} \, P(w \mid \theta_j)}_{\text{topic models}}$$

What PLSA Lacks

PLSA models **how words are generated given a document**: it specifies $P(w \mid d)$.

But each training document d gets its own coverage vector $\boldsymbol{\pi}_d = (\pi_{d1}, \ldots, \pi_{dk})$, **fitted to that specific document**. The model has no rule for where coverage vectors come from.

So PLSA never defines $P(d)$, a probability for whole documents:

- it explains the words of documents it has already seen
- it says nothing about how a **new** document would be generated

PLSA is a generative model for words, not for documents.

What PLSA Lacks

In practice, this causes three problems:

- **New documents**: a new review arrives, and PLSA has no coverage vector for it. We have to refit, or patch it with a heuristic.
- **No probability for a document**: we cannot ask how likely a held-out document is, so we cannot compute its perplexity.
- **Overfitting**: one coverage vector per training document means the number of parameters grows with the size of the corpus.

The fix: let the model **generate the coverage vectors too**, drawing each $\boldsymbol{\pi}_d$ from a shared distribution (a **Dirichlet**).

Latent Dirichlet Allocation (LDA)

LDA keeps the PLSA story and adds **one step**. To generate a document d:

0. **Draw its coverage vector $\boldsymbol{\pi}_d$** from a **Dirichlet** distribution.



Then, for every word of d, as in PLSA (without the background model):

1. pick a topic j with probability π_{dj}
2. pick a word from that topic, with probability $P(w \mid \theta_j)$

So the probability of a word is still $P(w \mid d) = \sum_{j=1}^{k} \pi_{dj} P(w \mid \theta_j)$.

The **Dirichlet** is a distribution over coverage vectors: it says which mixtures are likely (e.g. "mostly one or two topics") and which are rare. It is shared by all documents, so it can also produce a coverage vector for a new one.

LDA also draws each topic's word distribution θ_j from a Dirichlet, in the same way.

PLSA vs LDA

In both: topics are word distributions, $P(w \mid d)$ is the same mixture of topics (+ background model in PLSA), documents are BoW, and k is fixed in advance.

	PLSA	LDA
Coverage π_d	fitted to each training document	drawn from a shared Dirichlet
Can generate	words in known documents	documents and their words
New documents	no coverage vector: refit	inferred from the Dirichlet
Probability of d	not defined	defined
Size of the model	grows with the number of documents	fixed: topics and the Dirichlet
Common words	absorbed by the background model θ_B	no background model: remove stopwords first

Estimating LDA: The Intuition

Goal: learn the *topic mix of each document* and the *word mix of each topic*. A common way is **Gibbs sampling**: start from a random guess and improve it one word at a time.

1. Give every word in every document a **random topic**.
2. Take word w in document d and **re-pick its topic**, favoring topics that are:
   - already common in d
   - often given to w in the rest of the collection
3. Repeat for every word, many times, until the topics stop changing much.

At the end, every word has a topic, and both distributions are just **counts**:

$$\pi_{dj} = \frac{\text{words of } d \text{ with topic } j}{\text{words of } d} \qquad P(w \mid \theta_j) = \frac{\text{times } w \text{ has topic } j \text{ in the collection}}{\text{words with topic } j \text{ in the collection}}$$

In practice a small pseudo-count is added to every count, as in Laplace smoothing.

LDA: Dummy Example

Suppose LDA, run with $k = 2$ on a collection of sentences, has found two topics. Each sentence is described by the share of its words that belongs to each topic.

For a **new** sentence, LDA keeps the topics fixed and infers its coverage, guided by the shared Dirichlet.

Topic A, $P(w \mid \theta_A)$: *cat* 0.2, *dog* 0.2, *animals* 0.15, *fish* 0.15, *barking* 0.1, *howling* 0.1, …

Topic B, $P(w \mid \theta_B)$: *eat* 0.2, *fish* 0.2, *fruit* 0.15, *cheese* 0.15, *lunch* 0.1, *vegetable* 0.1, …

Train	Coverage $\boldsymbol{\pi}_d$
I eat fish and fruit	$\pi_A = 0$ $\pi_B = 1$
Fish are animals	$\pi_A = 1$ $\pi_B = 0$
My cat eats fish	$\pi_A = 0.33$ $\pi_B = 0.67$
…	…

Test	Coverage $\boldsymbol{\pi}_d$
My dog eats cheese	$\pi_A = 0.33$ $\pi_B = 0.67$

PLSA could not give a coverage to the new sentence without refitting the model.

Other Topic Modeling Methods

PLSA and LDA are probabilistic, but the same output (topics as weighted words, documents as mixtures of topics) can be obtained in other ways:

- **Latent Semantic Analysis (LSA)**: factorizes the document-term matrix with linear algebra (SVD). The ancestor of PLSA, but its weights can be **negative**, so the topics are hard to read.
- **Non-Negative Matrix Factorization (NMF)**: approximates the document-term matrix (often TF-IDF) as the product of a documents × topics and a topics × terms matrix, with **non-negative** weights only. Each document is an additive combination of topics, each topic an additive combination of terms: fast, and often as readable as LDA.
- **BERTopic**: embeds documents with BERT (later in the course), **clusters** the embeddings, then describes each cluster by its most distinctive words, with a class-based variant of TF-IDF. Topics capture meaning beyond shared words, but by default each document gets **one** topic.

Evaluating Topic Models

A topic model is a language model, so it can be evaluated like one:

- **Perplexity**: train on some documents, then measure how surprised the model is by **held-out documents**. Lower is better.

But a model that predicts words well does not necessarily produce topics that people find meaningful; the two can even disagree. So we also check the **topics themselves**:

- **Topic coherence**: do the top words of a topic occur in the same documents? *battery, charge, hours, last* is coherent; *battery, Paris, blue* is not.
- **Human judgment**: for example, **word intrusion**: hide a random word among a topic's top words and see whether people can spot it.

Topic vs Sentiment

Documents can share a topic and have opposite sentiments.

"the battery lasts forever" and *"the battery died in an hour"*

	Topic	**Sentiment**
Question	what is the text **about**?	how does the writer **feel** about it?
Carried by	content words, mostly nouns: *battery*, *screen*, *price*	adjectives, adverbs, negation, intensifiers: *great*, *not*, *very*
Typical learning	unsupervised (clustering, topic models), or supervised topic labels	supervised classifiers, or lexicons
Stopword removal	usually helps	can **destroy** the signal: *not good* $\rightarrow$ *good*
A single word flips it?	rarely	often: *not*, *never*, *but*

Topic and Sentiment Together

Many real questions need **both**:

- *"For a given product, what are the recurrent topics in the negative and in the positive reviews?"*: classify sentiment first, then run a topic model on each side separately.
- **Aspect-based sentiment**: find the aspects a review talks about, then the sentiment towards each one:

*"Great **screen**, but the **battery** is a joke."*

- screen: positive
- battery: negative

The topics say *what* is being judged, the sentiment says *how*.

Keeping Negation in a Bag of Words

A bag of words throws away word order, and with it *not*. A simple trick puts the negation back: prefix **NOT_** to every word after a negation, up to the next punctuation mark.

didn't like this movie , but $\rightarrow$ *didn't NOT_like NOT_this NOT_movie , but*

NOT_like is now a feature of its own, which the classifier can learn to weight as negative, while *like* keeps its positive weight.

The cost is a larger vocabulary: every word can now appear in two versions.

Sources

- Kumar, ch. 7: Clustering
- Zhai & Massung (2016), ch. 17: Topics